Team Perseus

Medusa | Tool: Dev Containers

1

Tool: Dev Containers extension for VS Code

- Tool: Dev Containers extension for VS Code https://marketplace.visualstudio.com/items?itemName=ms-vscode-remote.remote-containers, version 0.469.0
-Used with: VS Code 1.106, Docker Desktop, https://containers.dev
- Our config: https://github.com/CSCI-435-SE/medusa/tree/develop/.devcontainer: Node 22, Yarn 3.2.1, Postgres 16, Redis 7
- Tutorial video: https://youtu.be/8beFZ7r59EQ

2

The problem

Setting up our Medusa monorepo by hand takes many steps: the right Node version (tests fail on Node 24), Corepack for Yarn, a long build, and separate Postgres and Redis containers on specific ports. One missed step means failing tests and time lost debugging setup instead of code. Dev Containers define the whole environment in config files in the repo, so every teammate gets an identical, working setup with one click.

3

When to use it

- Projects with many setup steps or services such as databases and caches
- Teams on mixed Mac, Windows and Linux machines
- Onboarding: one click instead of a setup guide

When not to use it

- Low-RAM or low-disk; Docker Desktop is heavy.
- Small single-language projects where `npm install` is the whole setup

4

Cost and free-plan limits

- VS Code, the Dev Containers extension and the spec: free
- Docker Desktop: free for personal use, education and small businesses (under 250 employees and under $10M revenue); larger companies need a paid plan
- GitHub Codespaces runs the same config in the cloud: free personal accounts get a limited monthly quota (about 120 core-hours and 15 GB storage), with more on GitHub Pro or the Student Pack

Year/Quarter/Month

Setup

1. Install Docker Desktop (https://www.docker.com/products/docker-desktop/) and start it.

2. Install VS Code and the **Dev Containers** extension.

3. Add a `.devcontainer/` folder to the repo:

  - `docker-compose.yml`: an app container (`mcr.microsoft.com/devcontainers/typescript-node:22`) plus `postgres:16-alpine` and `redis:7-alpine`, all sharing one network so the databases are on `localhost`

  - `devcontainer.json`: `postCreateCommand` runs `corepack enable && yarn install && yarn build`; `remoteEnv` sets the `DB_*` variables; `forwardPorts` exposes 9000, 8000, 5173 and 6006

  - Tip: start from Command Palette > **Dev Containers: Add Dev Container Configuration Files** and pick a Node + PostgreSQL template

4. Command Palette > **Dev Containers: Reopen in Container**. The first build takes several minutes.

First result: in the container terminal, run a real integration test against the container's Postgres:

yarn workspace @medusajs/region test:integration # 18/18 passing

** Watch out for: port conflicts with services already on your laptop, published ports being open to your whole network unless bound to `127.0.0.1`, and files in the container's home folder being lost on rebuild.*